Contents

Abstract **1**

1 Introduction **1**

2 System Model and MPC Formulation **2**
2.1 Runge–Kutta Discretization 3

3 Problem 1: Terminal Cost and Terminal Set **3**
3.1 Implementation Summary 3
3.2 Local LQR Controller 4
3.3 Terminal Cost Matrix 4
3.4 Terminal Set Computation 5
3.5 Numerical Results 6
3.6 Visualization of the Terminal Ellipsoid 6
3.7 Design Choices 7

4 Problem 2: MPC with Terminal Cost and Terminal Constraint **8**
4.1 MPC Formulation Used in the Script 8
4.2 RK4 Discretization 9
4.3 Chosen Horizon Length 10
4.4 Closed-Loop MPC Algorithm 10
4.5 Initial Guess and Warm Start 11
4.6 Simulation Results 11
4.7 Design Choices 12

5 Problem 3: Short-Horizon MPC without Terminal Ingredients **13**
5.1 Main Difference Compared with Problem 2 13
5.2 Prediction and Simulation Setup 13
5.3 RK4 Prediction Model 14
5.4 Weighting Matrices Tested in the Script 14
5.5 Closed-Loop MPC Algorithm 15
5.6 Expected Closed-Loop Behaviour 16
5.7 Simulation Results 16

6 Problem 4: Short-Horizon MPC with Modified Stage Cost **18**
6.1 Motivation 18
6.2 Script Overview 18
6.3 Influence of Initial Guess 18
6.4 Simulation Results 19
6.5 Comparison with Problems 2 and 3 19
6.6 Possible Improvements 19

7 Conclusion **19**

A MATLAB Code Listings **20**
A.1 TerminalIngredients.m 20
A.2 DoublePendulum_P2.m 24
A.3 DoublePendulum_P3.m 28
A.4 DoublePendulum_P4.m 33

List of Figures

1 Projection of the terminal ellipsoid onto the angle plane. 6
2 Projection of the terminal ellipsoid onto the angular-velocity plane. 7
3 Closed-loop trajectory of the inverted double pendulum using terminal-constrained MPC. The plots show θ_1, θ_2, and the applied input u over the 4 s simulation. . . 12
4 Closed-loop simulation results for Problem 3 with four different weighting cases. All cases use $T = 1\,\mathrm{s}$, $T_{\mathrm{sim}} = 10\,\mathrm{s}$, and no terminal cost or terminal constraint. . . 17
5 Problem 4 closed-loop results 19

Listings

1 TerminalIngredients.m 20
2 DoublePendulum_P2.m 24
3 DoublePendulum_P3.m 29
4 DoublePendulum_P4.m 33

Abstract

This report documents a nonlinear model predictive control (MPC) strategy for the swing-up control of an inverted double pendulum. The system starts from the downward configuration

$$x_0 = \begin{bmatrix} \pi & \pi & 0 & 0 \end{bmatrix}^\mathsf{T}$$

and should be driven to the unstable upright equilibrium

$$x_f = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^\mathsf{T}.$$

The report is structured around four tasks: computation of terminal ingredients, implementation of MPC with terminal cost and terminal constraint, investigation of short-horizon MPC without terminal ingredients, and comparison with a modified stage cost that rewards upright configurations.

To complete after simulations

Add a concise summary of the final numerical findings: selected horizon length, terminal set value α, convergence behaviour, computation time, and the best-performing controller variant.

1 Introduction

The control task is a nonlinear swing-up problem for an inverted double pendulum. The first link is actuated by an electrical drive, while the second link is passive and mounted at the end of the first link. The state vector is

$$x(t) = \begin{bmatrix} \theta_1(t) & \theta_2(t) & \dot{\theta}_1(t) & \dot{\theta}_2(t) \end{bmatrix}^\mathsf{T}, \tag{1}$$

where θ_1 and θ_2 are measured relative to the vertical y-axis. The control input is the drive torque $u(t)$, subject to the hard input constraint

$$\mathcal{U} = \{u \in \mathbb{R} : |u| \leq 10\}. \tag{2}$$

The objective is to move the pendulum from the downward rest configuration to the upright equilibrium. This is challenging because the upright state is unstable, the model is nonlinear, and the second link is not directly actuated. MPC is appropriate for this task because it can handle nonlinear dynamics, input saturation, and finite-horizon optimal control objectives in a systematic way.

Documentation strategy

For the problem set, this report follows the same structure:

1) implementation objective,
2) explanation of the MATLAB script,
3) design choices,
4) numerical results,

2 System Model and MPC Formulation

The nonlinear system is represented in compact form as

$$\dot{x}(t) = f(x(t), u(t)), \tag{3}$$

where the function `NonlDynamics(x,u)` evaluates the state derivative. The pendulum parameters include the link masses m_1, m_2, inertias J_1, J_2, lengths L_1, L_2, center-of-mass distances l_1, l_2, gravitational acceleration g, and friction coefficients r_1, r_2.

The continuous-time MPC problem is

$$\min_{\bar{u}(\cdot;t)} \quad \int_t^{t+T} \left(\|\bar{x}(\tau;t)\|_Q^2 + \|\bar{u}(\tau;t)\|_R^2 \right) d\tau + \|\bar{x}(t+T;t)\|_P^2 \tag{4}$$

$$\text{s.t.} \quad \dot{\bar{x}} = f(\bar{x}, \bar{u}), \tag{5}$$

$$\bar{x}(t;t) = x(t), \tag{6}$$

$$\bar{u}(\tau;t) \in \mathcal{U}, \quad \forall \tau \in [t, t+T], \tag{7}$$

$$\bar{x}(t+T;t) \in \Omega_\alpha, \tag{8}$$

with terminal set

$$\Omega_\alpha = \{x \in \mathbb{R}^4 : x^\mathsf{T} P x \leq \alpha\}. \tag{9}$$

For the numerical implementation, the continuous dynamics are discretized using a fourth-order Runge–Kutta method with sampling time

$$\delta = 0.05\,\text{s}. \tag{10}$$

The number of prediction steps is computed as

$$N = \text{round}\left(\frac{T}{\delta}\right), \tag{11}$$

which avoids numerical inconsistencies caused by floating-point division.

2.1 Runge–Kutta Discretization

Given the state x_k and input u_k, the RK4 update is

$$k_1 = f(x_k, u_k), \tag{12}$$

$$k_2 = f\left(x_k + \frac{\delta}{2}k_1, u_k\right), \tag{13}$$

$$k_3 = f\left(x_k + \frac{\delta}{2}k_2, u_k\right), \tag{14}$$

$$k_4 = f(x_k + \delta k_3, u_k), \tag{15}$$

$$x_{k+1} = x_k + \frac{\delta}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{16}$$

The RK4 method is a suitable compromise between integration accuracy and computational cost. It is more accurate than forward Euler for the same sampling time, which is relevant because the double pendulum has strongly nonlinear transient behaviour during swing-up.

3 Problem 1: Terminal Cost and Terminal Set

Objective

The aim of this problem is to compute the terminal ingredients required for the MPC formulation of the inverted double pendulum. The required quantities are a local stabilizing controller K_{loc}, a terminal cost matrix P, and a terminal set parameter α for the ellipsoidal terminal region

$$\Omega_\alpha = \left\{x \in \mathbb{R}^4 : x^\top P x \leq \alpha\right\}. \tag{17}$$

The implementation is contained in `TerminalIngredients.m`.

3.1 Implementation Summary

The script first defines the physical parameters of the two-link pendulum, including the masses, lengths, inertias, friction coefficients, and gravitational acceleration. The center-of-mass distances and moments of inertia are computed from

$$l_1 = \frac{L_1}{2}, \qquad l_2 = \frac{L_2}{2}, \qquad J_1 = \frac{m_1 L_1^2}{12}, \qquad J_2 = \frac{m_2 L_2^2}{12}. \tag{18}$$

Using these parameters, the continuous-time linearized system matrices A and B around the upright equilibrium are assembled directly in the script.

The local auxiliary controller is designed using a continuous-time LQR. The state and input weights are chosen as

$$Q = \text{diag}(200, 200, 20, 20), \qquad R = 20. \tag{19}$$

The larger penalties on θ_1 and θ_2 reflect that angular displacement from the upright equilibrium is the dominant control objective near the origin. The angular velocity penalties are lower, which still damps the motion but avoids making the local controller unnecessarily aggressive. The

relatively large scalar value of R penalizes excessive input torque and is consistent with the input constraint $|u| \leq 10$.

3.2 Local LQR Controller

MATLAB's `lqr` function returns a gain K_{lqr} for the convention

$$u = -K_{\text{lqr}}x, \qquad A_{\text{cl}} = A - BK_{\text{lqr}}. \tag{20}$$

In the lecture notation used for quasi-infinite horizon MPC, the local feedback is written as

$$u = K_{\text{loc}}x, \qquad A_{\text{cl}} = A + BK_{\text{loc}}. \tag{21}$$

Therefore, the script explicitly sets

$$K_{\text{loc}} = -K_{\text{lqr}}. \tag{22}$$

This sign convention is important because the same numerical gain would destabilize the linearized system if inserted with the wrong sign.

For the submitted script, the resulting local feedback gain is

$$K_{\text{loc}} = \begin{bmatrix} 0.4827 & 62.8621 & 10.1295 & 14.6099 \end{bmatrix}. \tag{23}$$

With this convention, the eigenvalues of $A + BK_{\text{loc}}$ are approximately

$$\lambda(A + BK_{\text{loc}}) = \{-11.0622,\ -5.9232,\ -2.9047 + 0.1620i,\ -2.9047 - 0.1620i\}. \tag{24}$$

All eigenvalues have negative real parts, so the linearized closed loop is locally asymptotically stable.

3.3 Terminal Cost Matrix

The terminal cost is selected as a quadratic Lyapunov function

$$V_f(x) = x^{\mathsf{T}} P x. \tag{25}$$

The script computes P from the shifted Lyapunov equation

$$(A_K + \kappa I)^{\mathsf{T}} P + P\,(A_K + \kappa I) = -\left(Q + K_{\text{loc}}^{\mathsf{T}} R K_{\text{loc}}\right), \tag{26}$$

where $A_K = A + BK_{\text{loc}}$ and $\kappa = 0.95$. The shift by κ enforces a stronger local decrease condition than the unshifted Lyapunov equation. The script checks that

$$\kappa < -\max_i \operatorname{Re}\left(\lambda_i(A_K)\right), \tag{27}$$

which ensures that the shifted closed-loop matrix remains Hurwitz before solving Equation (26).

The computed terminal cost matrix is

$$P = \begin{bmatrix} 4549.6622 & 7951.5764 & 2333.6072 & 2178.6762 \\ 7951.5764 & 18542.4823 & 4851.3783 & 4874.5617 \\ 2333.6072 & 4851.3783 & 1333.6310 & 1300.1101 \\ 2178.6762 & 4874.5617 & 1300.1101 & 1293.7103 \end{bmatrix}. \quad (28)$$

The matrix is symmetrized numerically by replacing it with $(P + P^{\mathsf{T}})/2$. This is a useful implementation detail because numerical Lyapunov solvers can return a matrix with small asymmetries caused by floating-point round-off.

3.4 Terminal Set Computation

The terminal set is defined as the ellipsoid

$$\Omega_\alpha = \left\{ x \in \mathbb{R}^4 : x^{\mathsf{T}} P x \le \alpha \right\}. \quad (29)$$

The script computes α in two stages.

First, it determines the largest value allowed by the input constraint under the local feedback law. Since $u = K_{\text{loc}} x$, the worst-case input over the ellipsoid is

$$\max_{x^{\mathsf{T}} P x \le \alpha} |K_{\text{loc}} x| = \sqrt{\alpha K_{\text{loc}} P^{-1} K_{\text{loc}}^{\mathsf{T}}}. \quad (30)$$

Imposing $|u| \le 10$ gives

$$\alpha_1 = \frac{10^2}{K_{\text{loc}} P^{-1} K_{\text{loc}}^{\mathsf{T}}} \approx 107.4810. \quad (31)$$

This value only checks input feasibility. It does not yet guarantee that the nonlinear system satisfies the required Lyapunov decrease condition.

Second, the script searches for a less conservative terminal set by checking the nonlinear remainder

$$\phi(x) = f(x, K_{\text{loc}} x) - A_K x. \quad (32)$$

For a candidate value of α, the script solves nonconvex boundary maximization problems using `fmincon` from multiple initial directions. A bisection loop is then applied between 0 and α_1. If the maximum value violates the local decrease condition, the upper bound is reduced; otherwise, the lower bound is increased. This produces the final value

$$\alpha \approx 0.5589. \quad (33)$$

3.5 Numerical Results

Table 1: Terminal ingredients computed by `TerminalIngredients.m`.

Quantity	Value
K_{loc}	[0.4827, 62.8621, 10.1295, 14.6099]
α_1 from input constraint	107.4810
α after nonlinear decrease check	0.5589
P	See matrix above

3.6 Visualization of the Terminal Ellipsoid

The script also creates a standard ellipsoid representation using the shape matrix

$$S = \alpha P^{-1}. \tag{34}$$

This is required because the MATLAB ellipsoid toolbox represents ellipsoids in the form

$$\mathcal{E} = \{x : (x - c)^{\mathsf{T}} S^{-1} (x - c) \leq 1\}. \tag{35}$$

The center is chosen as $c = 0$ because the terminal set is centered at the upright equilibrium. The script plots two projections: one in the (θ_1, θ_2) plane and one in the $(\dot{\theta}_1, \dot{\theta}_2)$ plane. These plots are useful for interpreting the geometry of the terminal set, although the actual terminal region is four-dimensional.

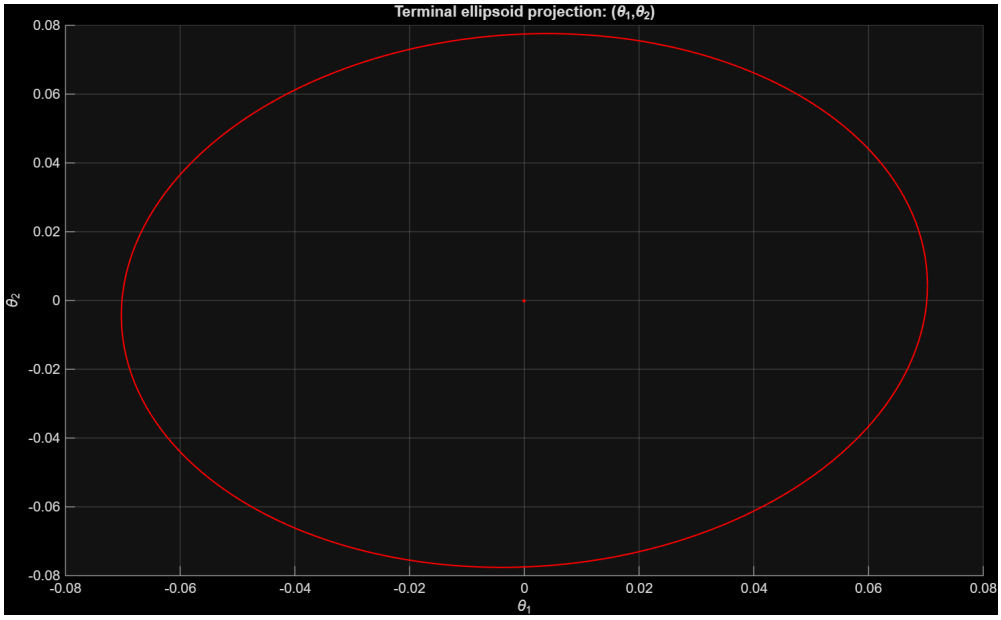


Figure 1: Projection of the terminal ellipsoid onto the angle plane.

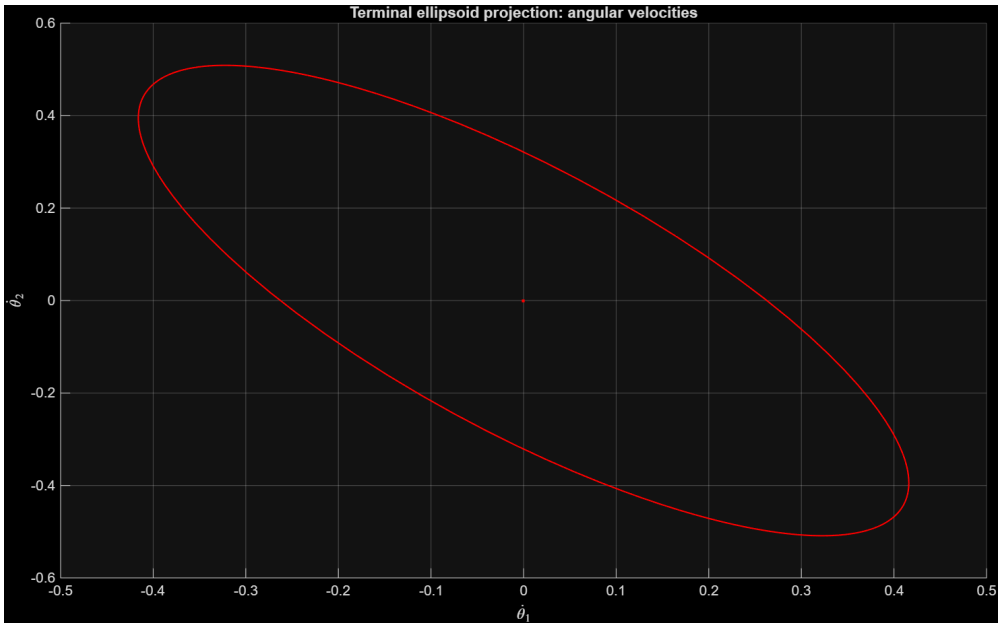


Figure 2: Projection of the terminal ellipsoid onto the angular-velocity plane.

3.7 Design Choices

The main design choice is to use an LQR controller for the linearized dynamics at the origin. This is appropriate because the terminal controller is only intended to operate locally, after the nonlinear MPC has brought the state sufficiently close to the equilibrium. The terminal set then acts like a handover region: outside this region, the nonlinear optimizer is responsible for the swing-up motion; inside this region, the LQR-based terminal ingredients guarantee local stabilizing behavior.

The use of $\kappa = 0.95$ makes the Lyapunov decrease condition stricter. This reduces the size of the admissible terminal set but improves the robustness of the local stability argument. The input-feasibility bound α_1 is much larger than the final α, which indicates that the nonlinear Lyapunov decrease condition is the active limitation, not the torque constraint.

The script uses multiple initializations for `fmincon` because the boundary search is nonconvex. This is a practical design choice: relying on one initial condition could miss a violating boundary point and return an unsafe terminal set.

4 Problem 2: MPC with Terminal Cost and Terminal Constraint

Objective

The aim of this problem is to implement the nonlinear MPC controller with the terminal ingredients from Problem 1 and simulate the closed loop for 4 s. The continuous-time double-pendulum model is discretized with a fourth-order Runge–Kutta method using the sampling time

$$\delta = 0.05\,\mathrm{s}. \tag{36}$$

The implementation is contained in `DoublePendulum_P2.m`; the full listing is provided in Listing 2.

4.1 MPC Formulation Used in the Script

The script starts by loading the terminal ingredients computed in Problem 1,

$$(K_{\mathrm{loc}}, P, \alpha) = \texttt{TerminalIngredients()}. \tag{37}$$

These values are then used in the terminal cost and terminal constraint of the finite-horizon MPC problem.

The state and input dimensions are defined as

$$n = 4, \qquad m = 1, \tag{38}$$

with the state vector

$$x = \begin{bmatrix} \theta_1 & \theta_2 & \dot{\theta}_1 & \dot{\theta}_2 \end{bmatrix}^{\mathsf{T}}. \tag{39}$$

The initial condition is

$$x_0 = \begin{bmatrix} \pi & \pi & 0 & 0 \end{bmatrix}^{\mathsf{T}}, \tag{40}$$

which corresponds to the downward hanging configuration. The control objective is to swing the system up and stabilize it at the origin.

The optimization variables are the predicted state trajectory and input sequence,

$$X = \{x_0, x_1, \ldots, x_N\}, \qquad U = \{u_0, u_1, \ldots, u_{N-1}\}. \tag{41}$$

In the MATLAB implementation, these are created using CasADi's `opti.variable` interface. The measured state is introduced as an optimization parameter `xmeasure`. This is a useful design choice because the same optimization problem structure can be reused at every MPC step; only the parameter value is updated.

The cost function implemented in the script is

$$J = \sum_{k=0}^{N-1} \delta \left(x_k^{\mathsf{T}} Q x_k + u_k^{\mathsf{T}} R u_k \right) + x_N^{\mathsf{T}} P x_N. \tag{42}$$

The stage-cost weighting matrices are

$$Q = \begin{bmatrix} 200 & 0 & 0 & 0 \\ 0 & 200 & 0 & 0 \\ 0 & 0 & 20 & 0 \\ 0 & 0 & 0 & 20 \end{bmatrix}, \qquad R = 20. \tag{43}$$

The angle states receive a stronger penalty than the angular velocities. This makes sense for the swing-up task because the main objective is to bring both pendulum links to the upright equilibrium, while the velocity terms mainly provide damping and prevent excessive oscillations.

4.2 RK4 Discretization

The nonlinear dynamics are integrated inside the prediction model using the fourth-order Runge–Kutta scheme. For a sample time δ, the discrete-time prediction step is

$$k_1 = f(x_k, u_k), \tag{44}$$

$$k_2 = f\left(x_k + \frac{\delta}{2}k_1, u_k\right), \tag{45}$$

$$k_3 = f\left(x_k + \frac{\delta}{2}k_2, u_k\right), \tag{46}$$

$$k_4 = f\left(x_k + \delta k_3, u_k\right), \tag{47}$$

$$x_{k+1} = x_k + \frac{\delta}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{48}$$

This RK4 step is used both in the CasADi optimization constraints and in the numerical helper function `rk4_step_numeric`. Using the same discretization in both places avoids a mismatch between the predicted trajectory and the simulated closed-loop update.

The equality constraint

$$x_{k+1} = F_\delta(x_k, u_k) \tag{49}$$

is imposed for every prediction step. The input constraint is implemented as

$$-10 \le u_k \le 10. \tag{50}$$

Finally, the terminal set constraint is imposed as

$$x_N^\top P x_N \le \alpha. \tag{51}$$

This constraint forces the optimizer to end the predicted trajectory inside the region where the local LQR-based terminal controller is valid.

4.3 Chosen Horizon Length

The submitted script uses

$$T = 3.0\,\text{s}, \qquad N = \text{round}\left(\frac{T}{\delta}\right) = 60. \tag{52}$$

The inline comments also identify 3.5 s and 4.0 s as practical fallback horizons if IPOPT reports infeasibility for 3.0 s. This is consistent with the nature of a swing-up problem: the optimizer must look far enough ahead to first inject energy into the system and then guide the final predicted state into the terminal ellipsoid.

Table 2: Prediction horizons considered for the terminal-constrained MPC implementation.

Horizon T [s]	Steps N	Interpretation
3.0	60	Implemented as the nominal horizon in the submitted script. This is computationally cheaper, but it may be close to the feasibility limit for reaching the terminal set.
3.5	70	Suggested fallback if the first optimization becomes infeasible. The additional prediction length gives the optimizer more room to complete the swing-up maneuver.
4.0	80	More conservative from a feasibility perspective, but more expensive because the number of decision variables and constraints increases.

Increasing T generally improves feasibility and closed-loop performance because the optimizer has more foresight. However, it also increases the computational burden. In this implementation, each additional 0.5 s adds ten shooting intervals because $\delta = 0.05\,\text{s}$. Since both state and input trajectories are optimized, this directly increases the size of the nonlinear program.

4.4 Closed-Loop MPC Algorithm

The receding-horizon structure used in the script is summarized in Algorithm 1. At every sampling instant, the optimization problem is solved, but only the first input is applied. The horizon is then shifted forward and the process is repeated.

Algorithm 1 Closed-loop MPC with terminal ingredients

Load K_{loc}, P, and α from `TerminalIngredients()`
Set $x(0) = x_0$, $\delta = 0.05\,\text{s}$, and $T_{\text{sim}} = 4\,\text{s}$
Choose $T = 3.0\,\text{s}$ and compute $N = \text{round}(T/\delta)$
Build the CasADi nonlinear program with RK4 dynamics, input constraints, terminal cost, and terminal set constraint
Initialize the input sequence with a sinusoidal excitation
for each simulation step k **do**
    Solve the nonlinear program with IPOPT
    Store the first optimized input $u_0^\star$ and the next closed-loop state
    Update the measured-state parameter
    Warm-start the next optimization by shifting the previous solution
    Append the terminal feedback input $u = K_{\text{loc}} x_N$ to the shifted input guess
end for

4.5 Initial Guess and Warm Start

A nonzero initial input guess is used for the first optimization,

$$u_{\text{init}}(k) = 8 \sin\left(\frac{2\pi k}{N}\right). \tag{53}$$

This is a relevant design choice for the swing-up task. A zero input sequence is usually a poor initial guess because it does not suggest an energy-building maneuver to the optimizer. The sinusoidal input provides a physically meaningful first trajectory that can help IPOPT find a feasible swing-up solution.

After each MPC solve, the script warm-starts the next optimization by shifting the previous optimal trajectories. The final appended input is computed using the local terminal feedback

$$u_{\text{terminal}} = K_{\text{loc}} x_N, \tag{54}$$

followed by saturation to satisfy the torque limit. This warm-start strategy is efficient because consecutive MPC problems are similar. It also makes the terminal part of the initial guess consistent with the terminal controller from Problem 1.

4.6 Simulation Results

Figure 3 shows the closed-loop response obtained with the terminal-constrained MPC formulation. The simulation was performed for $T_{\text{sim}} = 4\,\text{s}$ starting from the downward equilibrium, where both angles are initially close to 180°.

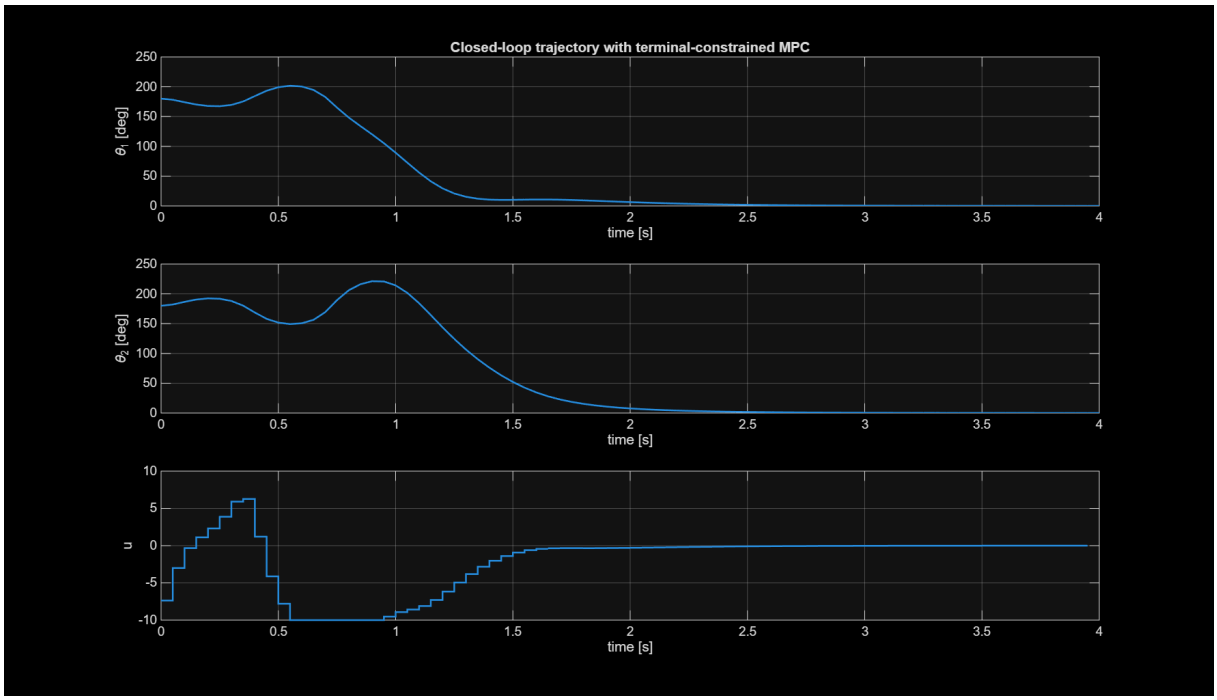


Figure 3: Closed-loop trajectory of the inverted double pendulum using terminal-constrained MPC. The plots show θ_1, θ_2, and the applied input u over the 4 s simulation.

The results show that both angular positions are successfully driven from the downward configuration toward the upright equilibrium. The motion is not monotonic, especially for θ_2, because the controller first has to inject energy into the system before stabilizing it around the origin. This is expected for a swing-up problem.

The input signal remains within the admissible bounds $u \in [-10, 10]$. During the initial part of the simulation, the input reaches the lower saturation bound. This indicates that the controller uses the available actuator authority aggressively to generate the required swing-up motion. After approximately 1.5 s, the input magnitude decreases and gradually approaches zero as the pendulum states move closer to the terminal region.

Overall, the terminal cost and terminal constraint improve the closed-loop behaviour by guiding the predicted state into a region where the local LQR controller is valid. Once the trajectory is close to the upright equilibrium, only small corrective inputs are required to keep the system stabilized.

4.7 Design Choices

Several implementation choices are important for this problem:

- **CasADi Opti formulation:** The script uses CasADi's symbolic optimization framework to construct the nonlinear program. This is suitable because the model is nonlinear and the constraints include a nonlinear terminal ellipsoid.
- **RK4 discretization:** RK4 is more accurate than forward Euler for the same sampling time. This matters because the swing-up trajectory is highly nonlinear.

- **Terminal cost and terminal set:** The terminal cost encourages the predicted final state to approach the origin, while the terminal set enforces entry into the local stability region.
- **Sinusoidal first initial guess:** The first solve is difficult because the system starts far from the upright equilibrium. A sinusoidal input gives the optimizer a better initial energy-injection pattern than a zero input.
- **Shifted warm start:** Reusing and shifting the previous solution reduces solver effort and improves numerical consistency between MPC iterations.
- **Terminal feedback appended to the guess:** Appending $K_{\text{loc}} x_N$ makes the tail of the warm start compatible with the terminal control law.

5 Problem 3: Short-Horizon MPC without Terminal Ingredients

Objective

The aim of this problem is to implement a nonlinear MPC controller without terminal cost and without terminal constraints. The prediction horizon is reduced to $T = 1\,\text{s}$ and the closed loop is simulated for $T_{\text{sim}} = 10\,\text{s}$. The stage cost is initially kept in the same form as in Problem 2 so that the effect of removing the terminal ingredients can be studied separately. The implementation is contained in `DoublePendulum_P3.m`; the full listing is provided in Listing 3.

5.1 Main Difference Compared with Problem 2

Problem 2 used three stabilizing components: the finite-horizon stage cost, the terminal cost $x_N^{\mathsf{T}} P x_N$, and the terminal constraint $x_N^{\mathsf{T}} P x_N \le \alpha$. In Problem 3, the last two components are removed. The optimization problem therefore only minimizes the accumulated stage cost

$$J = \sum_{k=0}^{N-1} \delta \left(x_k^{\mathsf{T}} Q x_k + u_k^{\mathsf{T}} R u_k \right), \tag{55}$$

subject to the nonlinear RK4 prediction model, the measured initial condition, and the input saturation limits

$$-10 \le u_k \le 10. \tag{56}$$

This makes the online optimization problem easier, but it also removes the explicit stability mechanism that forces the predicted state into a locally stabilizable terminal region.

5.2 Prediction and Simulation Setup

The script uses the same state and input dimensions as before,

$$n = 4, \qquad m = 1, \tag{57}$$

with the initial condition

$$x_0 = \begin{bmatrix} \pi & \pi & 0 & 0 \end{bmatrix}^{\mathsf{T}}. \tag{58}$$

The sampling time is

$$\delta = 0.05\,\mathrm{s}. \tag{59}$$

The prediction horizon is fixed to

$$T = 1\,\mathrm{s}, \qquad N = \mathrm{round}\left(\frac{T}{\delta}\right) = 20. \tag{60}$$

The use of `round` is a relevant implementation detail. It avoids numerical issues caused by floating-point division when converting the continuous prediction horizon into an integer number of shooting intervals.

The closed-loop simulation length is

$$T_{\mathrm{sim}} = 10\,\mathrm{s}, \qquad N_{\mathrm{sim}} = \mathrm{round}\left(\frac{T_{\mathrm{sim}}}{\delta}\right) = 200. \tag{61}$$

Thus, the nonlinear program is solved 200 times in closed loop. At each sampling instant, only the first optimized input is applied to the nonlinear plant.

5.3 RK4 Prediction Model

As in Problem 2, the nonlinear dynamics are discretized using a fourth-order Runge–Kutta method. For each prediction step,

$$k_1 = f(x_k, u_k), \tag{62}$$

$$k_2 = f\left(x_k + \frac{\delta}{2}k_1, u_k\right), \tag{63}$$

$$k_3 = f\left(x_k + \frac{\delta}{2}k_2, u_k\right), \tag{64}$$

$$k_4 = f\left(x_k + \delta k_3, u_k\right), \tag{65}$$

$$x_{k+1} = x_k + \frac{\delta}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{66}$$

The same RK4 formula is used in two places: inside the CasADi equality constraints and inside the numerical helper function `rk4_step_numeric`. This is a good design choice because the prediction model and the closed-loop plant update remain consistent.

5.4 Weighting Matrices Tested in the Script

The script implements a `weight_case` switch with four possible tuning cases. The script was executed for four different values of `weight_case`. This allows the influence of the weighting matrices Q and R on the short-horizon closed-loop behaviour to be compared systematically.

Table 3: Weighting cases implemented in `DoublePendulum_P3.m`.

Case	Q	R	Purpose
1	$\mathrm{diag}(200, 200, 20, 20)$	20	Baseline setting, equal to the stage cost used in Problem 2.
2	$\mathrm{diag}(200, 200, 20, 20)$	1	Cheaper input. This allows more aggressive actuation and can help the controller inject more energy.
3	$\mathrm{diag}(200, 200, 20, 20)$	100	Expensive input. This makes the controller more conservative and can reduce the ability to swing up.
4	$\mathrm{diag}(500, 500, 20, 20)$	20	Stronger angle penalty. This places greater emphasis on reducing θ_1 and θ_2.

The active case penalizes angular deviation more strongly than the baseline case. This is a reasonable tuning attempt because the main task is to bring both links from approximately 180° to 0°. However, for a short-horizon swing-up task, increasing the angle penalty alone does not necessarily guarantee convergence. The optimizer may still prefer locally attractive actions that reduce the one-second cost without planning a complete energy-building maneuver.

5.5 Closed-Loop MPC Algorithm

The receding-horizon implementation is summarized in Algorithm 2. The algorithm has the same basic MPC structure as in Problem 2, but the terminal cost and terminal constraint are absent.

Algorithm 2 Short-horizon MPC without terminal ingredients

1: Set $x(0) = x_0$, $\delta = 0.05\,\mathrm{s}$, $T = 1\,\mathrm{s}$, and $T_{\mathrm{sim}} = 10\,\mathrm{s}$
2: Compute $N = \mathrm{round}(T/\delta) = 20$
3: Select the weighting matrices using `weight_case`
4: Build the CasADi nonlinear program with RK4 dynamics and input constraints
5: Use a zero input sequence to generate the first initial state trajectory guess
6: **for** each simulation step k **do**
7: Solve the nonlinear program with IPOPT
8: Apply only the first optimal input $u_0^\star$
9: Simulate the nonlinear plant forward by one RK4 step
10: Update the measured-state parameter `xmeasure`
11: Warm-start the next optimization by shifting the previous predicted state and input trajectories
12: **end for**

The warm-start strategy is useful because consecutive MPC problems are similar. After one solve, the previous optimal state trajectory is shifted forward and the last state is propagated one additional step using the final input. The input guess is shifted in the same way, with the

last input repeated. This reduces the numerical effort compared with restarting IPOPT from zero at every sampling instant.

5.6 Expected Closed-Loop Behaviour

The closed loop is not guaranteed to converge to the origin in this problem. The main reason is that the terminal ingredients have been removed while the horizon has also been shortened to only one second. The double-pendulum swing-up task is strongly nonlinear and requires a sequence of actions that may temporarily move the state away from the origin in order to build enough energy. A short finite-horizon cost can fail to reward this long-term behaviour.

In other words, the controller can become myopic. It sees only the next one second and may choose inputs that look good over that short interval, but do not place the system on a trajectory that can be stabilized later. This is why Problem 2 used a longer horizon together with a terminal set: the terminal condition forces the prediction to end in a region where the local LQR controller can stabilize the system.

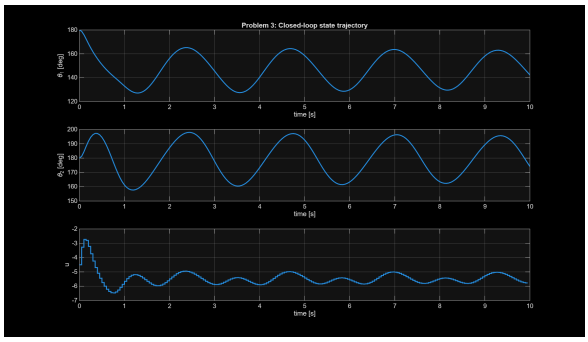
The weighting study shows the following qualitative tendencies:

- Reducing R makes actuation cheaper. This can help the controller apply stronger inputs, but it can also increase oscillations or input saturation.
- Increasing R makes actuation expensive. This often gives smoother inputs, but it can prevent the controller from injecting enough energy for swing-up.
- Increasing the angle penalties in Q makes the optimizer focus more strongly on reducing θ_1 and θ_2. This can improve local regulation, but it does not replace the missing terminal stability condition.

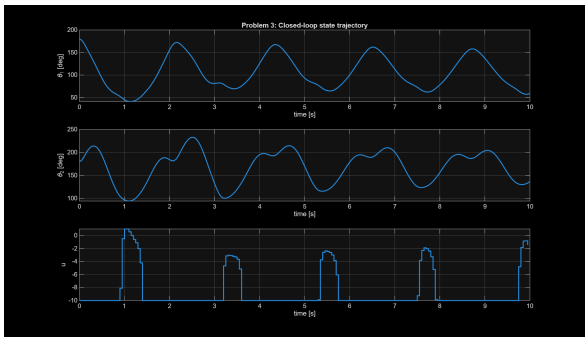
Therefore, tuning Q and R can change the transient behaviour, but it does not provide the same theoretical stabilizing effect as a terminal cost and terminal constraint.

5.7 Simulation Results

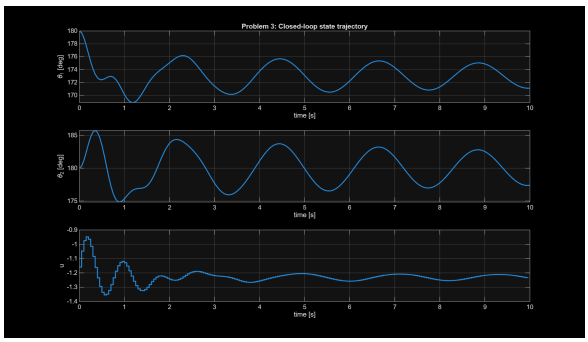
Four weighting cases were simulated in order to study the influence of the state and input penalties.



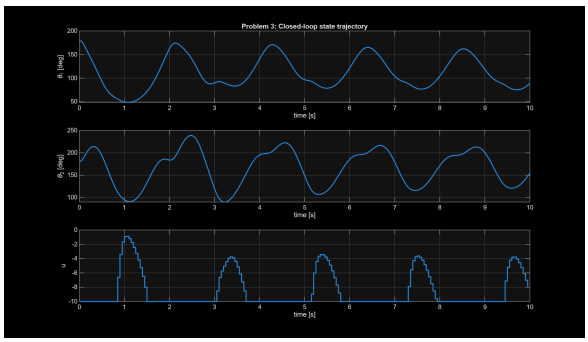
(a) Case 1: baseline weighting.



(b) Case 2: reduced input penalty.



(c) Case 3: increased input penalty.



(d) Case 4: increased angle penalty.

Figure 4: Closed-loop simulation results for Problem 3 with four different weighting cases. All cases use $T = 1\,\mathrm{s}$, $T_{\mathrm{sim}} = 10\,\mathrm{s}$, and no terminal cost or terminal constraint.

The four plots show that the closed-loop behaviour is strongly affected by the choice of Q and R. Since the controller uses a short prediction horizon and no terminal ingredients, convergence is not guaranteed. The weighting matrices can improve or worsen the transient response, but they do not replace the stabilizing role of the terminal cost and terminal constraint used in Problem 2.

The main observations are:

- **Case 1** uses the same stage cost as Problem 2, but without the terminal ingredients. This isolates the effect of removing the terminal cost and terminal set.
- **Case 2** reduces the input penalty. This allows more aggressive control action and can improve energy injection, but it may also cause stronger saturation or oscillations.
- **Case 3** increases the input penalty. This makes the input smoother and more conservative, but it can prevent the controller from applying enough torque for swing-up.
- **Case 4** increases the angular position penalties. This encourages the optimizer to reduce θ_1 and θ_2 more strongly, but it can still suffer from the short-horizon limitation.

6 Problem 4: Short-Horizon MPC with Modified Stage Cost

Objective

Implement a short-horizon MPC controller without terminal constraints using the modified stage cost

$$\|\bar{x}(\tau;t)\|_Q^2 + \|\bar{u}(\tau;t)\|_R^2 - 200\cos(\bar{\theta}_1(\tau;t)) - 200\cos(\bar{\theta}_2(\tau;t)). \tag{67}$$

The weighting matrices are

$$Q = \mathrm{diag}(0,0,1,1), \qquad R = 1. \tag{68}$$

6.1 Motivation

The modified cost directly rewards configurations where the links are close to the upright position. Since $\cos(0) = 1$, the terms $-200\cos(\theta_1)$ and $-200\cos(\theta_2)$ make the upright position attractive in the optimization objective. This helps compensate for the short horizon by giving the optimizer a stronger incentive to swing the pendulum upward.

6.2 Script Overview

This problem can be implemented by modifying `DoublePendulum_P3.m`. The terminal cost and terminal constraint remain removed, while the stage cost is changed. The optimizer should be tested with different initial guesses for the first optimization, for example

$$u_{\text{init}} = +\mathbf{1}, \qquad u_{\text{init}} = -\mathbf{1}. \tag{69}$$

6.3 Influence of Initial Guess

Because the optimization problem is nonlinear and nonconvex, the initial guess can affect the local optimum found by the solver. A positive initial input sequence and a negative initial input sequence may inject energy in different directions. Therefore, the closed-loop trajectory may differ even when the MPC parameters are unchanged.

Table 4: Effect of the first optimization initial guess. Replace the placeholder entries after simulation.

Initial guess	Closed-loop behaviour	Input usage	Converges?
$+\mathbf{1}$	TODO	TODO	TODO
$-\mathbf{1}$	TODO	TODO	TODO
Other	TODO	TODO	TODO

6.4 Simulation Results

Problem 4 closed-loop results

Export your MATLAB figure as `figures/problem4_closed_loop.pdf`.

Figure 5: Problem 4 closed-loop results

Script explanation to add

After you provide the Problem 4 script, add a paragraph explaining how the cosine terms were implemented, how the modified Q and R affect the optimizer, and how the chosen initial guesses influenced the result.

6.5 Comparison with Problems 2 and 3

Problem 2 uses terminal ingredients and is expected to give stronger stability guarantees near the origin. Problem 3 removes those ingredients and may fail with a short horizon because the optimizer does not sufficiently account for future stabilization. Problem 4 modifies the stage cost to make the upright configuration attractive even with a short horizon, but convergence is still not guaranteed because the problem remains nonlinear, constrained, and nonconvex.

6.6 Possible Improvements

Possible improvements include:

- using several initial guesses and selecting the best feasible solution,
- combining the cosine swing-up cost with a terminal stabilizing cost near the origin,
- switching from swing-up MPC to local LQR once the state enters a small neighbourhood of the origin,
- tuning the coefficient 200 in the cosine terms,
- adding robustness checks for different initial states and model parameter variations.

7 Conclusion

This report investigated nonlinear MPC strategies for the swing-up control of an inverted double pendulum. Terminal ingredients were used to improve local stability in the first controller. A shorter-horizon controller without terminal ingredients was then tested to reduce online

computation. Finally, a modified stage cost with cosine terms was used to encourage swing-up behaviour directly.

Final conclusion to complete

After all simulations are complete, replace this box with a concise conclusion answering:

- Which controller performed best?
- Which horizon length was selected and why?
- Did the short-horizon controller converge without terminal ingredients?
- How sensitive was the modified-cost MPC to the initial guess?
- What would be improved in a future implementation?

A MATLAB Code Listings

This appendix includes the MATLAB scripts used for the implementation. Replace the placeholder files in the `scripts/` folder with your final scripts. The listings update automatically when the project is recompiled.

A.1 TerminalIngredients.m

```
% This is the exercise file for the Studienleistung MPC, SoSe 2024,
% Problem 1.
% In this script, the terminal ingredients, i.e., a local controller, a
% terminal set and a terminal cost for an MPC scheme with terminal region
% constraint for swing-up control of an inverted double pendulum are computed.
% The linearized model of the pendulum and the necessary parameters for
% computation of the local controller are given.


function [K, P,alpha] = TerminalIngredients()

%% System paramter
m1 = 0.75;
m2 = 1;
g = 9.81;
L1 = 0.75;
L2 =1;
l1   = 1/2*L1;
l2   = 1/2*L2;
J1 = m1*L1^2/12;
J2 = m2*L2^2/12;
r1 = 0.02; r2 = 0.04;
p = J2*m2*L1^2 + m1*m2*l1^2*l2^2 + J2*m1*l1^2 + J1*m2*l2^2 + J1*J2;

%% Linearized model
A = [
        0,                          0,
            1,                                  0;
        0,                          0,
            0,                                  1;
```

```
(g*(m2*l2^2 + J2)*(L1*m2 + l1*m1))/p,                  -(L1*g*l2^2*m2^2)/p, -(r1
    *(m2*l2^2 + J2))/p,                  (L1*l2*m2*r2)/p;
     -(L1*g*l2*m2*(L1*m2 + l1*m1))/p, (g*l2*m2*(m2*L1^2 + m1*l1^2 + J1))/p,
                (L1*l2*m2*r1)/p, -(r2*(m2*L1^2 + m1*l1^2 + J1))/p];

B = [          0;
               0;
(m2*l2^2 + J2)/p;
   -(L1*l2*m2)/p];

n = size(B,1); m = size(B,2);


%% Design local controller, terminal cost, terminal set

%% Design parameters
Q = diag([200, 200, 20, 20]);
R = 20;
u_min = -10;
u_max =  10;
kappa = 0.95;

%% Step 1: Local Controller (LQR)
% MATLAB lqr gives u = -K_lqr*x and closed loop A - B*K_lqr.
% The QIH-MPC notation used in lecture uses u = K*x and closed loop A + B*K.

K  = -lqr(A, B, Q, R);
AK = A + B*K;

fprintf('K = \n'); disp (K)
fprintf('eig(A+BK) = \n'); disp(eig(AK))

%% Step 2: terminal cost matrix P
if kappa >= -max(real(eig(AK)))
    error('Choose a smaller kappa. Need kappa < -max(real(eig(A+B*K))).')
end

% For MATLAB lyap(A,Q): A*X + X*A'' + Q = 0.
% Require: (AK+kappa*I)'*P + P*(AK+kappa*I) = -(Q + K'*R*K).
P = lyap((AK + kappa*eye(n))', Q + K'*R*K);
P = 0.5*(P + P');      % numerical symmetrization

fprintf('P = \n'); disp(P)

%% Step 3: alpha_1 from the input constraint |Kx| <= 10
% max_{x'*P*x <= alpha} |Kx| = sqrt(alpha*K*inv(P)*K')
alpha_1 = min(u_max^2, u_min^2)/(K*(P\K'));

fprintf('alpha_1 = %.12g\n', alpha_1)

%% Step 4: alternative less-conservative terminal set computation
% Check the local Lyapunov decrease condition using fmincon and bisection.
```

```
% phi(x) = f(x,Kx) - (A+B*K)x is the nonlinear remainder.
phi = @(x) NonlDynamics(x, K*x) - AK*x;

alpha_lb = 0;
alpha_ub = alpha_1;
alpha    = alpha_1;
max_iter = 40;

% Multiple initializations are important because this is a nonconvex problem.
rng(1);
dir = [eye(n), -eye(n), randn(n,40)];

try
    opt = optimoptions('fmincon', ...
        'Algorithm','sqp', ...
        'Display','off', ...
        'MaxFunctionEvaluations',10000, ...
        'MaxIterations',10000, ...
        'OptimalityTolerance',1e-10, ...
        'StepTolerance',1e-10);
catch
    opt = optimset('Display','off','MaxFunEvals',10000,'MaxIter',10000);
end

for i = 1:max_iter
    fval_min = inf;

    % Cholesky factor for points on x'*P*x = alpha
    C = chol(P);

    for j = 1:size(dir,2)
        d = dir(:,j);
        d = d/norm(d);
        x0 = sqrt(alpha)*(C\d);

        [~, val] = fmincon(@(x) localDecreaseObjective(x,P,phi,kappa), ...
                           x0, [], [], [], [], [], [], ...
                           @(x) terminalSetConstraint(x,P,alpha), opt);

        fval_min = min(fval_min, val);
    end

    max_val = -fval_min;  % because we minimized the negative expression

    if max_val >= 0
        alpha_ub = alpha; % local decrease condition violated
    else
        alpha_lb = alpha; % local decrease condition satisfied
    end

    alpha = 0.5*(alpha_lb + alpha_ub);
end
```

```
131
132 alpha = alpha_lb;
133 fprintf('alpha = %.12g\n', alpha)
134
135 %% Step 5: Ellipsoid toolbox representation and visualization
136 % Terminal set:
137 % Omega_alpha = {x in R^4 : x'*P*x <= alpha}
138 %
139 % Standard ellipsoid form:
140 % E = {x : (x-c)'*inv(S)*(x-c) <= 1}
141 % with c = 0 and S = alpha*inv(P).
142
143 plot_terminal_ellipsoid = true;
144
145 if plot_terminal_ellipsoid
146     try
147         % Shape matrix of the terminal ellipsoid
148         S_terminal = alpha*(P\eye(n));
149
150         % Enforce exact numerical symmetry for Ellipsoids toolbox
151         S_terminal = 0.5*(S_terminal + S_terminal');
152
153         % Optional safety check
154         eig_S = eig(S_terminal);
155         fprintf('Eigenvalues of terminal ellipsoid shape matrix:\n');
156         disp(eig_S);
157
158         if min(eig_S) <= 0
159             warning('Terminal ellipsoid shape matrix is not positive definite.')
                ;
160         end
161
162         % Full 4D ellipsoid object
163         E_terminal = ellipsoid(zeros(n,1), S_terminal);
164
165         fprintf('Terminal ellipsoid shape matrix S = alpha*inv(P):\n');
166         disp(S_terminal);
167
168         %% Projection onto theta_1 - theta_2 plane
169         idx_theta = [1 2];
170         S_theta = S_terminal(idx_theta, idx_theta);
171         S_theta = 0.5*(S_theta + S_theta');
172
173         E_theta = ellipsoid(zeros(2,1), S_theta);
174
175         figure;
176         plot(E_theta);
177         grid on;
178         xlabel('\theta_1');
179         ylabel('\theta_2');
180         title('Terminal ellipsoid projection: (\theta_1,\theta_2)');
181
```

```
        %% Projection onto angular velocity plane
        idx_omega = [3 4];
        S_omega = S_terminal(idx_omega, idx_omega);
        S_omega = 0.5*(S_omega + S_omega');

        E_omega = ellipsoid(zeros(2,1), S_omega);

        figure;
        plot(E_omega);
        grid on;
        xlabel('$\dot{\theta}_1$', 'Interpreter','latex');
        ylabel('$\dot{\theta}_2$', 'Interpreter','latex');
        title('Terminal ellipsoid projection: angular velocities');

    catch ME
        warning('Ellipsoid toolbox visualization failed: %s', ME.message);
    end
end

end

%% Local helper functions
function cost = localDecreaseObjective(x,P,phi,kappa)
    denom = x'*P*x;
    if denom <= 1e-12
        value = -kappa;
    else
        value = (x'*P*phi(x) - kappa*denom)/denom;
    end
    cost = -value;
end

function [c, ceq] = terminalSetConstraint(x,P,alpha)
    c = x'*P*x - alpha;
    ceq = [];
end
```

Listing 1: TerminalIngredients.m

A.2 DoublePendulum_P2.m

```
% This is the exercise file for the Studienleistung MPC, SoSe 2026,
% Problem 2.
% In order to run this script, you need the toolboxes Casadi and Ellipsoid.
% Casadi is freely available at https://web.casadi.org/
% and Ellipsoid at https://code.google.com/archive/p/ellipsoids/downloads
% In the following, an MPC scheme with terminal region constraint is
% implemented for swing-up control of an inverted double pendulum.

clear; clc; close all;
import casadi.*

%% Terminal controller, cost, set
```

```
13
14 % Terminal controller, cost, set
15 [K_loc,P,alpha] = TerminalIngredients();
16 E = ellipsoid(alpha*inv([P(1,1) P(1,2); P(1,2) P(2,2)]));
17 figure(1)
18 plot(E); xlabel('theta_1 [rad]'); ylabel('theta_2 [rad]'); title('Terminal
       Region')
19
20
21 %% Optimization Problem
22
23 % System dimensions
24 n = 4;
25 m = 1;
26
27 % Sample time (discretization steps)
28 delta = 0.05;
29
30 % Horizon
31 % A terminal-constrained swing-up require a sufficiently long horizon.
32 % Testing with T = 3.0 s. IPOPT reports infeasibility, increase T=3.5 or 4.0.
33 T = 3.0;                    % continuous prediction horizon [s]
34 N = round(T/delta);         % discrete prediction horizon
35
36 % Simulation time
37 Tsim = 4; % (continuous)
38 mpciterations = round(Tsim/delta); % (discrete)
39
40 % Initial conditions
41 x0 = [pi; pi; 0; 0];
42
43 % Closed loop trajectory
44 x_cl = zeros(n,mpciterations+1);
45 u_cl = zeros(m,mpciterations);
46
47 x_cl(:,1) = x0;
48
49 % Stage cost (quadratic)
50 Q = diag([200,200,20,20]);
51 R = 20;
52
53 % Set up Optimization problem
54 opti = casadi.Opti();
55 % IPOPT settings. Increase print level while debugging, if needed.
56 p_opts = struct('expand',true);
57 s_opts = struct('max_iter',4000, ...
58                 'print_level',0, ...
59                 'tol',1e-6, ...
60                 'acceptable_tol',1e-5);
61 opti.solver('ipopt',p_opts,s_opts);
62
63
```

```
% Define optimization variables
x = opti.variable(n,N+1);       % predicted states: x(:,k)
u = opti.variable(m,N);         % predicted inputs: u(:,k)
% Parameter for the measured state x(t)
xmeasure = opti.parameter(n,1);
opti.set_value(xmeasure,x0);


% Implement constraints and cost functions
J = 0;

for k = 1:N
    % RK4 discretization of xdot = NonlDynamics(x,u)
    k1 = NonlDynamics(x(:,k),                 u(:,k));
    k2 = NonlDynamics(x(:,k) + delta/2*k1,    u(:,k));
    k3 = NonlDynamics(x(:,k) + delta/2*k2,    u(:,k));
    k4 = NonlDynamics(x(:,k) + delta*k3,      u(:,k));
    x_next = x(:,k) + delta/6*(k1 + 2*k2 + 2*k3 + k4);

    opti.subject_to(x(:,k+1) == x_next);

    % Discretized integral cost
    J = J + delta*(x(:,k)'*Q*x(:,k) + u(:,k)'*R*u(:,k));
end

% Initial state constraint
opti.subject_to(x(:,1) == xmeasure);

% Input constraints
u_min = -10;
u_max = 10;

opti.subject_to(u >= u_min);
opti.subject_to(u <= u_max);

% Terminal set constraint: x_N' P x_N <= alpha
opti.subject_to(x(:,N+1)'*P*x(:,N+1) <= alpha);

% Terminal cost
J = J + x(:,N+1)'*P*x(:,N+1);
opti.minimize(J);

% Provide an initial guess

% Initial input guess. The sine excitation gives the optimizer a nonzero
% swing-up candidate.Checking if the first solve fails, will try init_sign = -1.
init_sign = 1;
u_init = init_sign*8*sin(2*pi*(0:N-1)/N);
opti.set_initial(u,u_init);


% Initial state guess obtained by simulating the nonlinear model with u_init.
```

```
x_init = zeros(n,N+1);
x_init(:,1) = x0;
for k = 1:N
    x_init(:,k+1) = rk4_step_numeric(x_init(:,k),u_init(:,k),delta);
end
opti.set_initial(x,x_init);


for k = 1:mpciterations % Loop over sampling time steps

    % Solve optimization problem and recover solution
    sol = opti.solve();
    x_OL = sol.value(x);
    u_OL = sol.value(u);

    % Update closed-loop trajectory
    x_cl(:,k+1) = x_OL(:,2);
     u_cl(:,k) = u_OL(:,1);

    % Update initial guess and constraints
    opti.set_value(xmeasure,x_cl(:,k+1));
    % Update initial guess by shifting the previous optimal solution.
    % The final appended input is the local terminal feedback.
    u_terminal = K_loc*x_OL(:,end);
    u_terminal = min(10,max(-10,u_terminal));
    x_terminal = rk4_step_numeric(x_OL(:,end),u_terminal,delta);
    opti.set_initial(x,[x_OL(:,2:N+1),x_terminal]);
    opti.set_initial(u,[u_OL(:,2:N),u_terminal]);


    % Plots
    figure(2)
    subplot(3,1,1) % Trajectories xw
    xlabel('time [s]');ylabel('theta_1 [deg]');
    plot(delta*k,x_cl(1,k+1)*180/pi,'bo')
    hold on
    plot(delta*(k-1):delta:delta*(k+N-1),x_OL(1,:)*180/pi,'r--')

    subplot(3,1,2) % Trajectories theta
    xlabel('time [s]');ylabel('theta_2 [deg]');
    plot(delta*k,x_cl(2,k+1)*180/pi,'bo')
    hold on
    plot(delta*(k-1):delta:delta*(k+N-1),x_OL(2,:)*180/pi,'r--')

    subplot(3,1,3) % Trajectories u
    xlabel('time [s]');ylabel('u');
    plot(delta*(k-1),u_cl(1,k),'bo')
    hold on
    plot(delta*(k-1):delta:delta*(k+N-2),u_OL(1,:),'r--')
    drawnow

    drawpendulum(x_cl(1,k),x_cl(2,k)); % Draws the double pendulum
```

```
168 end
169
170 %% Final documentation plot
171
172 time_x = 0:delta:Tsim;
173 time_u = 0:delta:(Tsim-delta);
174
175 figure(4)
176 subplot(3,1,1)
177 plot(time_x,x_cl(1,:)*180/pi,'LineWidth',1.2); grid on
178 xlabel('time [s]'); ylabel('\theta_1 [deg]')
179 title('Closed-loop trajectory with terminal-constrained MPC')
180
181 subplot(3,1,2)
182 plot(time_x,x_cl(2,:)*180/pi,'LineWidth',1.2); grid on
183 xlabel('time [s]'); ylabel('\theta_2 [deg]')
184
185 subplot(3,1,3)
186 stairs(time_u,u_cl(1,:),'LineWidth',1.2); grid on
187 xlabel('time [s]'); ylabel('u')
188
189 saveas(figure(4),'Problem2_closed_loop.png')
190
191 %% Local helper functions
192 function x_next = rk4_step_numeric(x,u,delta)
193     k1 = NonlDynamics(x,                u);
194     k2 = NonlDynamics(x + delta/2*k1,   u);
195     k3 = NonlDynamics(x + delta/2*k2,   u);
196     k4 = NonlDynamics(x + delta*k3,     u);
197
198     x_next = x + delta/6*(k1 + 2*k2 + 2*k3 + k4);
199 end
200
201 function drawpendulum(theta1,theta2)
202 % This function draws the double pendulum, at the current angels theta1 and
      theta2
203     f = figure(3);
204     f.Position(3:4) = [550 520];
205     L1 = 0.75;
206     L2 = 1;
207     x1 = L1*sin(theta1); y1 = L1*cos(theta1);
208     x2 = x1 + L2*sin(theta2); y2 = y1 + L2*cos(theta2);
209     plot([0,x1,x2],[0,y1,y2],'o-','LineWidth',2.5,'color',[0 .447 .741], '
          MarkerFaceColor',[0 .447 .741]);
210     xlim([-2,2])
211     ylim([-2,2])
212     axis square
213 end
```

Listing 2: DoublePendulum_P2.m

A.3 DoublePendulum_P3.m

```
% This is the exercise file for the Studienleistung MPC, SoSe 2026,
% Problem 3.
% In order to run this script, you need the Casadi Toolbox which is
% freely available at https://web.casadi.org/
% In the following, an MPC scheme without terminal constraints is
% implemented for swing-up control of an inverted double pendulum.


clear; clc; close all;
import casadi.*

%% Optimization Problem

% System dimensions
n = 4;
m = 1;

% Sample time (discretization steps)
delta = 0.05;

% Horizon
T = 1; % (continuous)
N = round(T/delta); % (discrete)

% Simulation time
Tsim = 10; % (continuous)
mpciterations = round(Tsim/delta); % (discrete)

% Initial conditions
x0 = [pi; pi; 0; 0];

% Closed loop trajectory
x_cl = zeros(n,mpciterations+1);
u_cl = zeros(m,mpciterations);
x_cl(:,1) = x0;


% Set up Optimization problem
opti = casadi.Opti();

% Define optimization variables
x = opti.variable(n,N+1);
u = opti.variable(m,N);

% Parameter for the measured state x(t)
xmeasure = opti.parameter(n,1);
opti.set_value(xmeasure,x0);

% Implement constraints and cost functions

% Weighting matrices
weight_case = 4;
```

```
53
54 switch weight_case
55     case 1
56         % Baseline: same stage cost as Problem 2
57         Q = diag([200, 200, 20, 20]);
58         R = 20;
59
60     case 2
61         % Cheaper input: more aggressive control
62         Q = diag([200, 200, 20, 20]);
63         R = 1;
64
65     case 3
66         % Expensive input: more conservative control
67         Q = diag([200, 200, 20, 20]);
68         R = 100;
69
70     case 4
71         % Stronger angle penalty
72         Q = diag([500, 500, 20, 20]);
73         R = 20;
74 end
75
76 % Finite-horizon cost.
77 % Problem 3 uses only the stage cost, i.e. no terminal cost.
78 J = 0;
79
80 % RK4 dynamics and stage cost
81 for i = 1:N
82
83     k1 = NonlDynamics(x(:,i),                u(:,i));
84     k2 = NonlDynamics(x(:,i) + delta/2*k1,   u(:,i));
85     k3 = NonlDynamics(x(:,i) + delta/2*k2,   u(:,i));
86     k4 = NonlDynamics(x(:,i) + delta*k3,     u(:,i));
87
88     x_next = x(:,i) + delta/6*(k1 + 2*k2 + 2*k3 + k4);
89
90     opti.subject_to(x(:,i+1) == x_next);
91
92     J = J + delta*(x(:,i)'*Q*x(:,i) + u(:,i)'*R*u(:,i));
93 end
94
95 % Input constraints
96 opti.subject_to(u >= -10);
97 opti.subject_to(u <= 10);
98
99 % Initial condition constraint
100 opti.subject_to(x(:,1) == xmeasure);
101
102 % Cost function
103 opti.minimize(J);
104
```

```
% Solver settings
opts = struct;
opts.ipopt.print_level = 0;
opts.print_time = false;
opts.ipopt.max_iter = 1000;
opti.solver('ipopt', opts);

% Initial guess
u_init = zeros(m,N);
x_init = zeros(n,N+1);
x_init(:,1) = x0;

for i = 1:N
    x_init(:,i+1) = rk4_step_numeric(x_init(:,i),u_init(:,i),delta);
end

opti.set_initial(x,x_init);
opti.set_initial(u,u_init);


for k = 1:mpciterations % Loop over sampling time steps

    % Solve optimization problem and recover solution
    sol = opti.solve();
    x_OL = sol.value(x);
    u_OL = sol.value(u);

    % Apply only the first input of the optimal sequence
    u_cl(:,k) = u_OL(:,1);

    % Simulate the nonlinear plant for one sampling step
    x_cl(:,k+1) = rk4_step_numeric(x_cl(:,k),u_cl(:,k),delta);

    % Update measured initial state for the next MPC problem
    opti.set_value(xmeasure,x_cl(:,k+1));

    % Warm start IPOPT by shifting the previous optimal solution forward
    x_last = rk4_step_numeric(x_OL(:,end),u_OL(:,end),delta);

    x_guess = [x_OL(:,2:N+1), x_last];
    u_guess = [u_OL(:,2:N),   u_OL(:,N)];

    opti.set_initial(x,x_guess);
    opti.set_initial(u,u_guess);

    % Plots
    figure(2)
    subplot(3,1,1) % theta_1 trajectory
    xlabel('time [s]');ylabel('theta_1 [deg]');
    plot(delta*k,x_cl(1,k+1)*180/pi,'bo')
    hold on
    plot(delta*(k-1):delta:delta*(k+N-1),x_OL(1,:)*180/pi,'r--')
```

```
157
158     subplot(3,1,2) %Trajectories theta
159     xlabel('time [s]');ylabel('theta_2 [deg]');
160     plot(delta*k,x_cl(2,k+1)*180/pi,'bo')
161     hold on
162     plot(delta*(k-1):delta:delta*(k+N-1),x_OL(2,:)*180/pi,'r--')
163
164     subplot(3,1,3) % Trajectories u
165     xlabel('time [s]');ylabel('u');
166     plot(delta*(k-1),u_cl(1,k),'bo')
167     hold on
168     plot(delta*(k-1):delta:delta*(k+N-2),u_OL(1,:),'r--')
169     drawnow
170
171     drawpendulum(x_cl(1,k+1),x_cl(2,k+1)); % Draws the double pendulum
172 end
173 %% Final closed-loop plots for documentation
174 time_x = 0:delta:Tsim;
175 time_u = 0:delta:Tsim-delta;
176
177 figure;
178 subplot(3,1,1)
179 plot(time_x,x_cl(1,:)*180/pi,'LineWidth',1.5)
180 grid on
181 xlabel('time [s]')
182 ylabel('\theta_1 [deg]')
183 title('Problem 3: Closed-loop state trajectory')
184
185 subplot(3,1,2)
186 plot(time_x,x_cl(2,:)*180/pi,'LineWidth',1.5)
187 grid on
188 xlabel('time [s]')
189 ylabel('\theta_2 [deg]')
190
191 subplot(3,1,3)
192 stairs(time_u,u_cl(1,:),'LineWidth',1.5)
193 grid on
194 xlabel('time [s]')
195 ylabel('u')
196
197 saveas(gcf,'Problem3_closed_loop_case4.png')
198
199 %% Helper functions
200 function drawpendulum(theta1,theta2)
201 % This function draws the double pendulum, at the current angels theta1 and
      theta2
202     f = figure(3);
203     f.Position(3:4) = [550 520];
204     L1 = 0.75;
205     L2 = 1;
206     x1 = L1*sin(theta1); y1 = L1*cos(theta1);
207     x2 = x1 + L2*sin(theta2); y2 = y1 + L2*cos(theta2);
```

```
    plot([0,x1,x2],[0,y1,y2],'o-','LineWidth',2.5,'color',[0 .447 .741], '
        MarkerFaceColor',[0 .447 .741]);
    xlim([-2,2])
    ylim([-2,2])
    axis square
end
function x_next = rk4_step_numeric(x,u,delta)
% Numeric RK4 step for initial guesses and warm-starting.
    k1 = full(NonlDynamics(x,              u));
    k2 = full(NonlDynamics(x+delta/2*k1,   u));
    k3 = full(NonlDynamics(x+delta/2*k2,   u));
    k4 = full(NonlDynamics(x+delta*k3,     u));
    x_next = x + delta/6*(k1 + 2*k2 + 2*k3 + k4);
end
```

Listing 3: DoublePendulum_P3.m

A.4 DoublePendulum_P4.m

```
%% DoublePendulum_P4.m
% Placeholder file.
% Replace this file with your final implementation for Problem 4.
%
% Expected task:
%   - nonlinear MPC without terminal constraints
%   - modified stage cost:
%       x'Qx + u'Ru - 200*cos(theta1) - 200*cos(theta2)
%   - Q = diag([0 0 1 1]) and R = 1
%   - test initial guesses u_init = +ones(m,N) and u_init = -ones(m,N)

% TODO: paste your final script here.
```

Listing 4: DoublePendulum_P4.m

References

[1] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.

[2] E. F. Camacho and C. Bordons, *Model Predictive Control*, 2nd ed. Springer, 2013.

[3] Institute of Automatic Control, *MPC – Swing up control of an inverted double pendulum*, Exercise sheet provided for the project, 2026.